DFA-Based Lexical Analyzer for Java Source Code

Programming Languages Course Assignment Report

---

1 Purpose and Scope

In this work, a DFA-based lexical analyzer was developed in Java. It reads Java source code character by character, identifies the lexemes and matches each lexeme with a token type. The analyzer reports unrecognized characters and invalid lexemes together with their line and column, and displays the resulting lexemes and tokens as a table. The recognized token types are listed in Table 1.

Table 1. Token types

Token type	Recognizing DFA	Example lexemes
`KEYWORD`	Figure 1 and Figure 2	`int`, `while`, `_`
`IDENTIFIER`	Figure 1	`count`, `$price2`
`INTEGER_LITERAL`	Figure 3 (q_1)	`0`, `42`
`FLOAT_LITERAL`	Figure 3 (q_2, q_5)	`3.14`, `12.`, `1.5e-3`
`BOOLEAN_LITERAL`	Figure 1 and Figure 2	`true`, `false`
`NULL_LITERAL`	Figure 1 and Figure 2	`null`
`OPERATOR`	Figure 4 and Figure 5	`==`, `++`, `&&`
`SEPARATOR`	Figure 6	`(`, `;`, `.`
`UNRECOGNIZED_CHAR`	No DFA can start	`#`, `@`
`INVALID_LEXEME`	Figure 3, non-accepting result	`2e+`, `3abc`

Out of scope: String and character literals, comments; hexadecimal and binary notation, type suffixes (`L`, `f`, `d`) and the digit separator; shift operators (`<<`, `>>`), `^`, `~`, `?`, `:`, `->`; the separators `@`, `...` and `::`.

2 DFA Designs

Notation: A single circle denotes a state and a double circle an accepting state; the start state is marked by an incoming arrow. Transitions that are not drawn lead to a dead state. *letter*: A–Z, a–z and other Unicode letters; *digit*: 0–9.

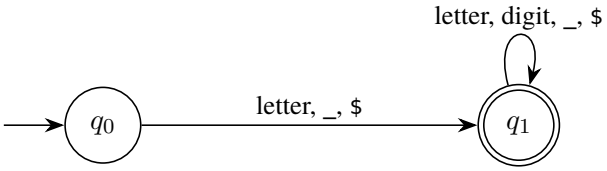


Figure 1. Identifier DFA

Recognizes variable, method and class names. Since a Java identifier cannot start with a digit, digits appear only on the self-loop at q_1. An accepted lexeme is then passed through the DFA in Figure 2 to distinguish it from reserved words.

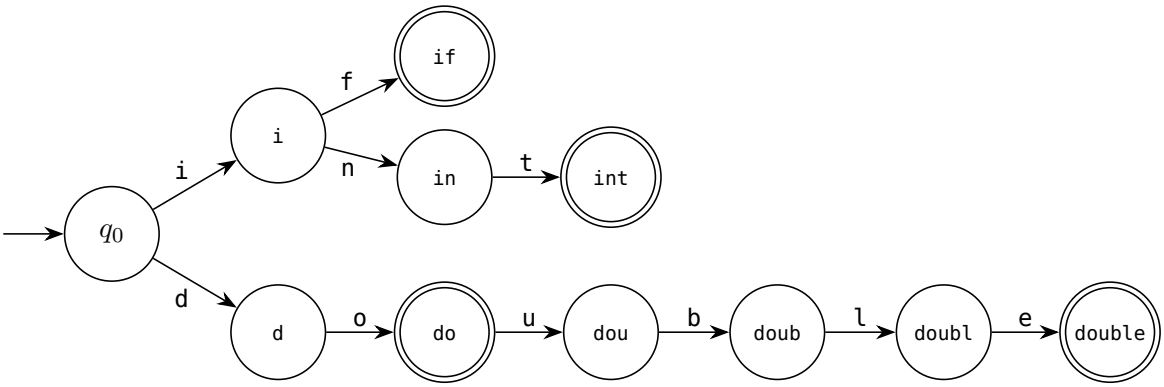


Figure 2. Reserved word (keyword) DFA

A prefix tree built from Java's 51 reserved words (including _); the figure shows the fragment for `if`, `int`, `do` and `double`. If the whole lexeme accepted by the identifier DFA is consumed and the DFA ends in an accepting state, the lexeme is labeled `KEYWORD`; otherwise it is labeled `IDENTIFIER` (e.g. `in`, `integer`). `true`, `false` and `null` are separated in the same way as `BOOLEAN_LITERAL` and `NULL_LITERAL`.

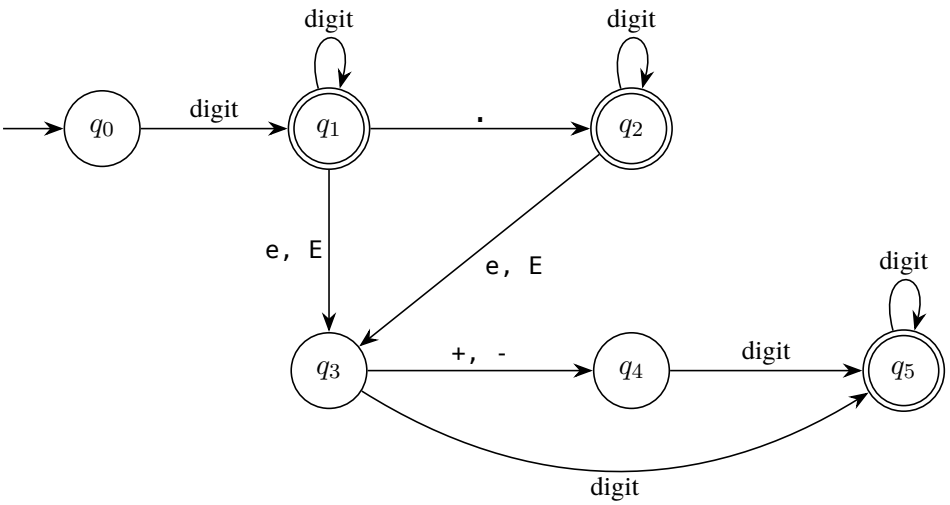


Figure 3. Numeric literal DFA

Recognizes integer and floating-point literals. Accepting states: $q_1 \rightarrow$ `INTEGER_LITERAL`; q_2 and $q_5 \rightarrow$ `FLOAT_LITERAL` (e.g. `42`, `3.14`, `12.`, `2e3`, `1.5e-3`). If the DFA stops in q_3 or q_4 (e.g. `2e+`), or a letter immediately follows the number (e.g. `3abc`), the lexeme is reported as invalid.

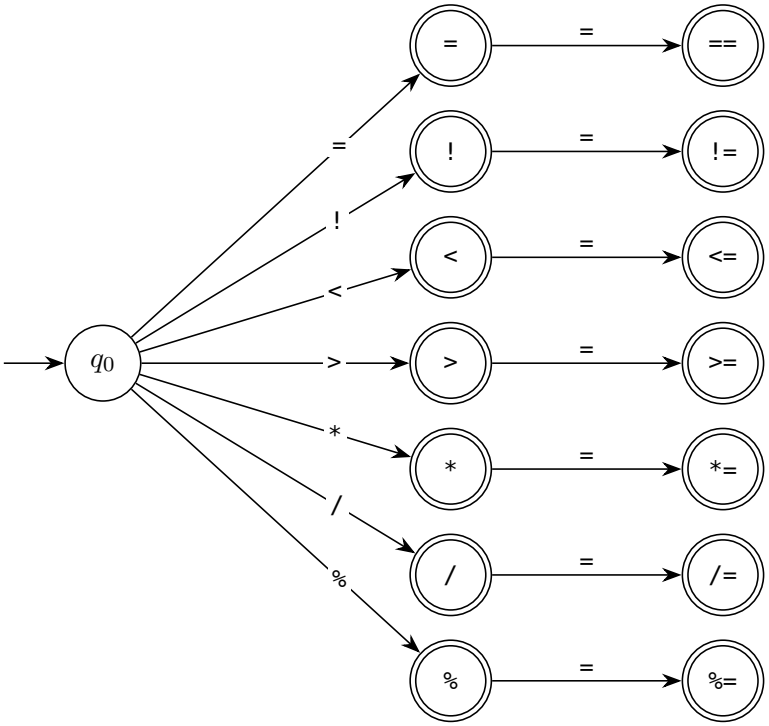


Figure 4. Operator DFA (I)

Recognizes assignment, comparison and arithmetic operators. If `=` follows the first character, a two-character operator (`==`, `!=`, `<=`, `>=`, `*=`, `/=`, `%=`) is read; otherwise the single-character operator is accepted.

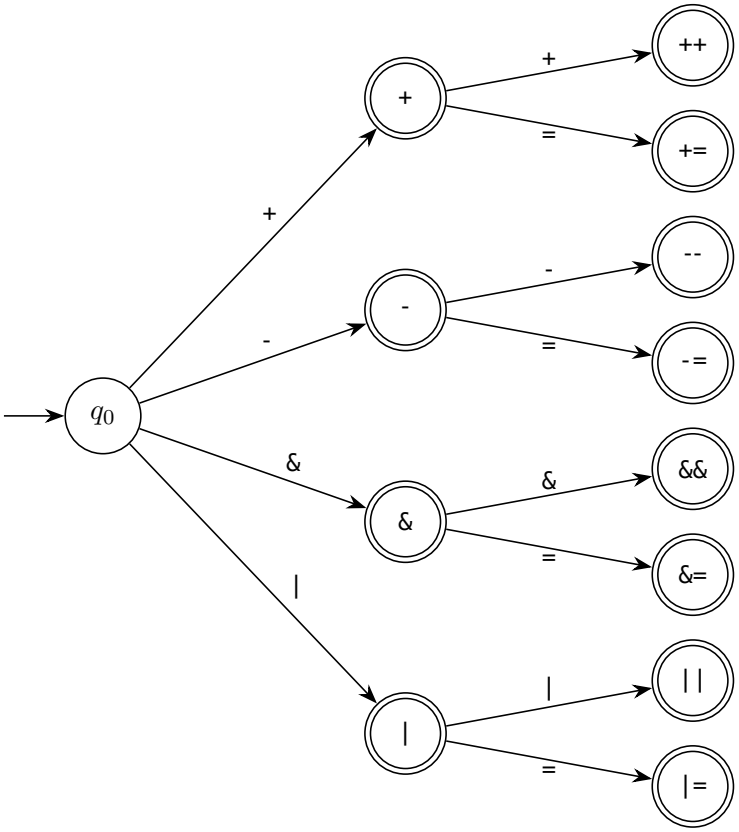


Figure 5. Operator DFA (II)

Recognizes increment/decrement, compound assignment and logical operators. If the first character is repeated, `++`, `--`, `&&` or `||` is read; if `=` follows, `+=`, `-=`, `&=` or `|=` is read.

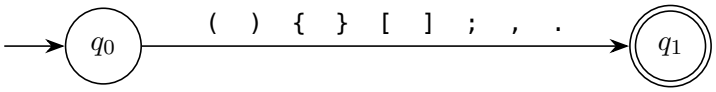


Figure 6. Separator DFA

Recognizes single-character separators; each one produces a separate `SEPARATOR` token.

3 Implementation

3.1 Program structure

The analyzer consists of a single file (`LexicalAnalyzer.java`). Each DFA extends the common abstract class `DFA`: the transition function δ is implemented as the method `transition(state, c)`, and the set of accepting states F as the method `isAccepting(state)`. For undefined transitions, `transition` returns -1 (dead state). The state numbers in the code match the q indices in the figures (Listing 1).

Table 2. Figures and corresponding classes

Figure	Class	Start characters
Figure 1	`IdentifierDFA`	letter, `_`, `$`
Figure 2	`ReservedWordDFA`	Runs on the identifier lexeme
Figure 3	`NumberDFA`	digit
Figure 4	`OperatorDFA1`	`=` `!` `<` `>` `*` `/` `%`
Figure 5	`OperatorDFA2`	`+` `-` `&` `|`
Figure 6	`SeparatorDFA`	`(` `)` `{` `}` `[` `]` `;` `,` `.`

Since the start-character sets are disjoint, at most one DFA can start with any given character.

Listing 1. Transition function of the DFA in Figure 3 (`NumberDFA.transition`)

```
int transition(int state, char c) {
    switch (state) {
        case 0:
            if (isDigit(c)) return 1;
            break;
        case 1:
            if (isDigit(c)) return 1;
            if (c == '.') return 2;
            if (c == 'e' || c == 'E') return 3;
            break;
        case 2:
            if (isDigit(c)) return 2;
            if (c == 'e' || c == 'E') return 3;
            break;
        case 3:
            if (c == '+' || c == '-') return 4;
            if (isDigit(c)) return 5;
            break;
        case 4:
            if (isDigit(c)) return 5;
            break;
        case 5:
            if (isDigit(c)) return 5;
            break;
    }
    return DEAD_STATE;
}
```

3.2 Character-by-character reading

The source file is read in a single pass from start to end. In each step, the `analyze` method does the following:

1. If the current character is a space, tab or line break, it is skipped and the line and column counters are updated.
2. The DFA that has a start transition on the character is found (Table 2). If there is none, the character is recorded as an unrecognized character (`UNRECOGNIZED_CHAR`) and scanning moves on to the next character.
3. The DFA is run character by character until no transition is left (Listing 2). This yields the longest match; in the input `a++`, `++` is read as a single lexeme.
4. If the DFA stopped in an accepting state, the lexeme is recorded with the token type of that final state (Table 1). A lexeme accepted by the identifier DFA is also passed through the DFA in Figure 2.
5. Scanning resumes right after the lexeme. The character that had no transition is not consumed, so it becomes the first character of the next lexeme; in the input `x=10`, the `=` character is not appended to `x`.

Listing 2. Method that runs a DFA character by character (`DFA.run`)

```
Result run(String text, int start) {
    int state = 0;                      // start in q0
    int i = start;
    while (i < text.length()) {
        int next = transition(state, text.charAt(i));
        if (next == DEAD_STATE) {       // no transition -> stop
            break;
        }
        state = next;
        i++;
    }
    return new Result(i - start, state);
}
```

3.3 Error reporting

Each error is written both to the lexeme/token table and to the error report, together with the line and column where the lexeme starts. The analysis does not stop at an error; it continues with the next character. The reported error types are given in Table 3.

Table 3. Error types and conditions

Error type	Condition	Example
`UNRECOGNIZED_CHAR`	A character that matches no DFA's start transition	`#`
`INVALID_LEXEME`	The DFA stops in a non-accepting state (Figure 3: q_3, q_4)	`2e+`
`INVALID_LEXEME`	A letter, _ or $ immediately follows a number; the adjacent characters are absorbed into the lexeme	`3abc`

3.4 Output

The program prints three sections: the **lexeme/token table** (sequence number, line, column, lexeme, token), the **error report** (location, lexeme and reason) and a **summary** (number of lexemes per token type). The output for the test input is given in Section 4.

4 Testing and Results

4.1 Test input

The analyzer was run on the file `sample_input.txt` shown in Listing 3. The file was prepared to contain every token type and the multi-character operators; three errors were deliberately placed on lines 31–33.

Listing 3. Test input (`sample_input.txt`)

```
public class Calculator {
    public static void main(String[] args) {
        int count = 0;
        double rate = 3.14;
        double small = 1.5e-3;
        double whole = 12.;
        boolean done = false;
        Object empty = null;

        while (count <= 10 && !done) {
            count++;
            rate += 0.5;
            if (count == 5 || rate >= 4.0) {
                done = true;
            }
            if (count != 3) {
                rate -= 1;
            }
        }

        int diff = count - 10;
        diff--;
        diff *= 2;
        int[] arr = new int[5];
        arr[0] = diff * 2 % 3;
        if (rate > 2.5 && diff < 0) {
            rate = rate / 2;
        }
        System.out.println(count);

        double err1 = 2e+;
        int err2 = 3abc;
        int err3 = 5 # 2;
    }
}
```

4.2 Lexeme and token table

A total of 173 lexemes were obtained from the test input (Table 4). In error rows, the token type is shown in bold.

Table 4. Lexeme and token table

No	Line	Col.	Lexeme	Token	No	Line	Col.	Lexeme	Token
1	1	1	public	KEYWORD	51	10	28	&&	OPERATOR
2	1	8	class	KEYWORD	52	10	31	!	OPERATOR
3	1	14	Calculator	IDENTIFIER	53	10	32	done	IDENTIFIER
4	1	25	{	SEPARATOR	54	10	36	)	SEPARATOR
5	2	5	public	KEYWORD	55	10	38	{	SEPARATOR
6	2	12	static	KEYWORD	56	11	13	count	IDENTIFIER
7	2	19	void	KEYWORD	57	11	18	++	OPERATOR
8	2	24	main	IDENTIFIER	58	11	20	;	SEPARATOR
9	2	28	(	SEPARATOR	59	12	13	rate	IDENTIFIER
10	2	29	String	IDENTIFIER	60	12	18	+=	OPERATOR
11	2	35	[	SEPARATOR	61	12	21	0.5	FLOAT_LITERAL
12	2	36	]	SEPARATOR	62	12	24	;	SEPARATOR
13	2	38	args	IDENTIFIER	63	13	13	if	KEYWORD
14	2	42	)	SEPARATOR	64	13	16	(	SEPARATOR
15	2	44	{	SEPARATOR	65	13	17	count	IDENTIFIER
16	3	9	int	KEYWORD	66	13	23	==	OPERATOR
17	3	13	count	IDENTIFIER	67	13	26	5	INTEGER_LITERAL
18	3	19	=	OPERATOR	68	13	28	||	OPERATOR
19	3	21	0	INTEGER_LITERAL	69	13	31	rate	IDENTIFIER
20	3	22	;	SEPARATOR	70	13	36	>=	OPERATOR
21	4	9	double	KEYWORD	71	13	39	4.0	FLOAT_LITERAL
22	4	16	rate	IDENTIFIER	72	13	42	)	SEPARATOR
23	4	21	=	OPERATOR	73	13	44	{	SEPARATOR
24	4	23	3.14	FLOAT_LITERAL	74	14	17	done	IDENTIFIER
25	4	27	;	SEPARATOR	75	14	22	=	OPERATOR
26	5	9	double	KEYWORD	76	14	24	true	BOOLEAN_LITERAL
27	5	16	small	IDENTIFIER	77	14	28	;	SEPARATOR
28	5	22	=	OPERATOR	78	15	13	}	SEPARATOR
29	5	24	1.5e-3	FLOAT_LITERAL	79	16	13	if	KEYWORD
30	5	30	;	SEPARATOR	80	16	16	(	SEPARATOR
31	6	9	double	KEYWORD	81	16	17	count	IDENTIFIER
32	6	16	whole	IDENTIFIER	82	16	23	!=	OPERATOR
33	6	22	=	OPERATOR	83	16	26	3	INTEGER_LITERAL
34	6	24	12.	FLOAT_LITERAL	84	16	27	)	SEPARATOR
35	6	27	;	SEPARATOR	85	16	29	{	SEPARATOR
36	7	9	boolean	KEYWORD	86	17	17	rate	IDENTIFIER
37	7	17	done	IDENTIFIER	87	17	22	-=	OPERATOR
38	7	22	=	OPERATOR	88	17	25	1	INTEGER_LITERAL
39	7	24	false	BOOLEAN_LITERAL	89	17	26	;	SEPARATOR
40	7	29	;	SEPARATOR	90	18	13	}	SEPARATOR
41	8	9	Object	IDENTIFIER	91	19	9	}	SEPARATOR
42	8	16	empty	IDENTIFIER	92	21	9	int	KEYWORD
43	8	22	=	OPERATOR	93	21	13	diff	IDENTIFIER
44	8	24	null	NULL_LITERAL	94	21	18	=	OPERATOR
45	8	28	;	SEPARATOR	95	21	20	count	IDENTIFIER
46	10	9	while	KEYWORD	96	21	26	-	OPERATOR
47	10	15	(	SEPARATOR	97	21	28	10	INTEGER_LITERAL
48	10	16	count	IDENTIFIER	98	21	30	;	SEPARATOR
49	10	22	<=	OPERATOR	99	22	9	diff	IDENTIFIER
50	10	25	10	INTEGER_LITERAL	100	22	13	--	OPERATOR

Table 4 (continued)

No	Line	Col.	Lexeme	Token	No	Line	Col.	Lexeme	Token
101	22	15	;	SEPARATOR	138	26	37	{	SEPARATOR
102	23	9	diff	IDENTIFIER	139	27	13	rate	IDENTIFIER
103	23	14	*=	OPERATOR	140	27	18	=	OPERATOR
104	23	17	2	INTEGER_LITERAL	141	27	20	rate	IDENTIFIER
105	23	18	;	SEPARATOR	142	27	25	/	OPERATOR
106	24	9	int	KEYWORD	143	27	27	2	INTEGER_LITERAL
107	24	12	[	SEPARATOR	144	27	28	;	SEPARATOR
108	24	13	]	SEPARATOR	145	28	9	}	SEPARATOR
109	24	15	arr	IDENTIFIER	146	29	9	System	IDENTIFIER
110	24	19	=	OPERATOR	147	29	15	.	SEPARATOR
111	24	21	new	KEYWORD	148	29	16	out	IDENTIFIER
112	24	25	int	KEYWORD	149	29	19	.	SEPARATOR
113	24	28	[	SEPARATOR	150	29	20	println	IDENTIFIER
114	24	29	5	INTEGER_LITERAL	151	29	27	(	SEPARATOR
115	24	30	]	SEPARATOR	152	29	28	count	IDENTIFIER
116	24	31	;	SEPARATOR	153	29	33	)	SEPARATOR
117	25	9	arr	IDENTIFIER	154	29	34	;	SEPARATOR
118	25	12	[	SEPARATOR	155	31	9	double	KEYWORD
119	25	13	0	INTEGER_LITERAL	156	31	16	err1	IDENTIFIER
120	25	14	]	SEPARATOR	157	31	21	=	OPERATOR
121	25	16	=	OPERATOR	158	31	23	2e+	**INVALID_LEXEME**
122	25	18	diff	IDENTIFIER	159	31	26	;	SEPARATOR
123	25	23	*	OPERATOR	160	32	9	int	KEYWORD
124	25	25	2	INTEGER_LITERAL	161	32	13	err2	IDENTIFIER
125	25	27	%	OPERATOR	162	32	18	=	OPERATOR
126	25	29	3	INTEGER_LITERAL	163	32	20	3abc	**INVALID_LEXEME**
127	25	30	;	SEPARATOR	164	32	24	;	SEPARATOR
128	26	9	if	KEYWORD	165	33	9	int	KEYWORD
129	26	12	(	SEPARATOR	166	33	13	err3	IDENTIFIER
130	26	13	rate	IDENTIFIER	167	33	18	=	OPERATOR
131	26	18	>	OPERATOR	168	33	20	5	INTEGER_LITERAL
132	26	20	2.5	FLOAT_LITERAL	169	33	22	#	**UNRECOGNIZED_CHAR**
133	26	24	&&	OPERATOR	170	33	24	2	INTEGER_LITERAL
134	26	27	diff	IDENTIFIER	171	33	25	;	SEPARATOR
135	26	32	<	OPERATOR	172	34	5	}	SEPARATOR
136	26	34	0	INTEGER_LITERAL	173	35	1	}	SEPARATOR
137	26	35	)	SEPARATOR					

4.3 Error report

Table 5. Error report

Line	Column	Lexeme	Error type	Description
31	23	2e+	Invalid lexeme	expected a digit after the exponent sign
32	20	3abc	Invalid lexeme	a letter immediately follows a number; identifiers cannot start with a digit
33	22	#	Unrecognized character	no DFA can start with this character

4.4 Summary

Table 6. Number of lexemes per token type

Token type	Count
KEYWORD	21
IDENTIFIER	38
INTEGER_LITERAL	15
FLOAT_LITERAL	6
BOOLEAN_LITERAL	2
NULL_LITERAL	1
OPERATOR	33
SEPARATOR	54
UNRECOGNIZED_CHAR	1
INVALID_LEXEME	2
Total	**173**

4.5 Additional tests

Longest match, reserved word recognition and error cases were also tested with short inputs (Table 7).

Table 7. Additional test inputs and results

Input	Produced lexemes (token)	What it shows
a+++b	a (IDENTIFIER), ++ (OPERATOR), + (OPERATOR), b (IDENTIFIER)	Longest match: ++ first, then +
x==5;	x (IDENTIFIER), == (OPERATOR), 5 (INTEGER_LITERAL), ; (SEPARATOR)	Lexeme boundaries without whitespace
int integer in	int (KEYWORD), integer (IDENTIFIER), in (IDENTIFIER)	Only an exact match is a KEYWORD
_ = $price2;	_ (KEYWORD), = (OPERATOR), $price2 (IDENTIFIER), ; (SEPARATOR)	_ is reserved; name starting with $
1. 1.e3 9E+2	1. (FLOAT_LITERAL), 1.e3 (FLOAT_LITERAL), 9E+2 (FLOAT_LITERAL)	Valid floating-point forms in Java
café <= 10	café (IDENTIFIER), <= (OPERATOR), 10 (INTEGER_LITERAL)	Identifier with a Unicode letter
3e x	3e (INVALID_LEXEME), x (IDENTIFIER)	q_3 in Figure 3 is not accepting
a @ b	a (IDENTIFIER), @ (UNRECOGNIZED_CHAR), b (IDENTIFIER)	No DFA starts with @

5 Conclusion

The developed analyzer recognizes all the token types and multi-character operators required by the assignment by means of DFAs. All 173 lexemes in the test input were matched with the correct token types, and the three deliberately placed errors were reported with the correct line and column. Table 8 shows where each requirement is met.

Table 8. Requirements coverage

	Requirement	**Where it is met**
1	DFA design for identifiers, keywords, numeric literals, operators and separators	Section 2
2	State transition diagrams	Figures 1–6
3	Implementing the DFAs in Java	Section 3.1, Table 2
4	Reading the source code character by character	Section 3.2, Listing 2
5	Multi-character operators (==, !=, <=, >=, ++)	Figures 4–5, Table 4
6	Reporting unrecognized characters and invalid lexemes	Section 3.3, Table 5
7	Displaying the lexemes and tokens as a table	Table 4

Project files: `LexicalAnalyzer.java` (source code), `sample_input.txt` (test input).

Mert Zeki Öztürk
Student No: 240260115